
System-Aware Compression and Edge Deployment of LLaMA Models

Shawal Latif Taimoor Karim

Abstract

Large Language Models (LLMs) such as LLaMA-2-7B have strong reasoning and text generation capabilities. However, deploying them on resource-constrained hardware remains challenging due to their large memory footprint, high inference latency. Running LLaMA-2-7B in FP16 precision requires over 25 GB of storage and nearly 15 GB of memory, making deployment on CPU-only systems and embedded platforms such as Raspberry Pi impractical. This project addresses that gap and we present a deployment-oriented compression pipeline. This pipeline integrates exact unstructured pruning, parameter-efficient fine-tuning, activation-aware quantization, and system-level inference support. We applied unstructured pruning with a target of 30% parameter reduction. Additionally we mitigated the resulting performance degradation using LoRA-based fine-tuning. We ensure stable low bit inference therefore we employed SmoothQuant to suppress the effect of activation before doing fine tuning we faced several issues for example exact pruning produces non-uniform layer dimensions across the transformer blocks and these are not supported by llama.cpp. We address the issue by modifying llama.cpp to dynamically infer per-layer dimensions at runtime directly from GGUF tensor metadata, enabling seamless inference without altering model formats or breaking backward compatibility. Towards the end we evaluated our approach on LLaMA-2-7B using WikiText-2 perplexity, inference throughput, latency, memory consumption, and model size. Exact 30% pruning improved CPU inference throughput from 36.8 to 45.9 tokens per second with modest perplexity degradation. We combined with quantization and the final compressed models achieve up to a 91.7% reduction in disk size (25.1 GB to 2.1 GB) and reduce peak memory usage from 15.0 GB to 2.4 GB. These results demonstrate that billion-parameter LLMs can be deployed effectively on CPUs and edge devices i.e. Raspberry Pi. All code is available at <https://github.com/tkarim45/LLM-Compression>.

1. Introduction

Large Language Models (LLMs) have now been widely used for natural language understanding and generation. They achieved remarkable results across tasks such as question answering, reasoning, translation, and dialogue. Models with billions of parameters, including LLaMA-2-7B, exhibit strong generalization and reasoning abilities, but their practical deployment remains limited by substantial computational and memory requirements. In FP16 precision, LLaMA-2-7B requires over 25 GB of storage and nearly 15 GB of peak memory during inference, making deployment infeasible on CPU-only systems and embedded devices such as Raspberry Pi.

Inference based frameworks like llama.cpp have shown that LLM inference without GPUs is possible efficient C/C++ implementations, SIMD optimizations, and memory-mapped model loading. However, inference performance on CPUs remains constrained by model size, memory bandwidth, and numerical precision. As a result, compression techniques are important for enabling real-time or near real-time inference on resource-constrained hardware.

Model compression for LLMs rely on pruning, quantization, or a combination of both. What pruning does is very interesting. It reduces the number of parameters by completely removing the less important weights or the redundant weights. Quantization reduces numerical precision to decrease memory usage and improve computational efficiency. Each technique has been studied individually and independently, deploying pruned and quantized models in real-world inference systems introduces additional challenges related to architectural assumptions and framework compatibility. One key particular issue we faced was that unstructured pruning results in non uniform layer dimensions across transformer blocks, violating the assumption of uniform architecture used by most inference engines, including standard llama.cpp.

In this work, we address these challenges by presenting a deployment-oriented compression pipeline that jointly considers algorithmic compression techniques and system-level inference constraints. To stabilize low-bit inference, we incorporate SmoothQuant, an activation-aware quantization technique that suppresses activation outliers prior to quantization. Furthermore, we resolved a critical incompatibility

issue between exact pruning and llama.cpp by changing the inference framework to dynamically infer per-layer dimensions directly from GGUF tensor metadata at runtime. This enables seamless inference of non-uniformly pruned models without changing the model format or breaking backward compatibility.

2. Literature Review

CPU-Based Inference and Deployment Frameworks.

Initially large language model pipelines assumed that GPUs were available due to the computational intensity of transformer architectures. Recent work contradicts this and has shown that LLMs can be executed efficiently on CPUs through low-level optimization and memory-aware design. The llama.cpp framework introduced a pure C/C++ implementation of LLaMA-style models with support for SIMD instructions, multi-threaded execution, and memory-mapped model loading, enabling inference without GPU acceleration (Gerganov, 2023). While this new method significantly improves accessibility, inference performance still remains constrained by model size, memory bandwidth, and numerical precision on resource-limited hardware.

It doesn't stop here, to further improve memory efficiency, the ggml and later GGUF tensor formats were introduced. These formats made life easier as they provided compact storage, fast loading, and reduced runtime overhead through memory mapping (Hugging Face, 2023). In addition to all of this, they reduced model loading time and peak memory usage but did not reduce the computational complexity of transformer inference. This still remains a bottleneck and as a result, CPU-based deployment of billion-parameter models remains impractical without additional compression techniques.

Post-Training Quantization for Large Language Models.

Now we introduce the concept of quantization. Quantization has emerged as a key technique for reducing the memory footprint and computational cost of LLM inference. Keeping this in mind it is to be considered that naive low-bit quantization often leads to severe accuracy degradation as there exist many activation outliers in transformer networks. AWQ showed that activation-aware weight quantization can significantly improve low-bit inference by identifying and preserving outlier channels during quantization (Lin et al., 2024). In conclusion all this showed the importance of handling activation distributions when quantizing large models.

Building on this concept, SmoothQuant proposed an activation-weight smoothing mechanism that redistributes quantization difficulty between activations and weights, enabling stable INT8 inference with minimal accuracy loss (Xiao et al., 2022). SmoothQuant is particularly useful for deployment scenarios for one main reason. It does not

require modifications to the inference graph and can be applied post-training. There were also subsequent methods such as SpinQuant and OmniQuant that further improved quantization accuracy using learned rotations and omnidirectional calibration, respectively (Liu et al., 2024; Shao et al., 2023). There is still one issue that these approaches introduce more training or there is severe calibration complexity. This limits their practicality for deployment pipelines.

Pruning and Structural Redundancy in LLMs. Now moving beyond quantization, new studies have shown that large language models contain a substantial amount of structural redundancy. LLM-Pruner demonstrated that unstructured and structured pruning can reduce model size by removing less important parameters while retaining competitive performance (Ma et al., 2023). More importantly, the LLM pruner applies layer-wise adaptive pruning which resulted in uniform reductions across transformer blocks.

While pruning is effective in reducing parameter count, it introduces architectural challenges during deployment. Exact pruning changes tensor dimensions across layers, breaking the assumption of uniform architecture commonly used by inference frameworks such as llama.cpp. As a result, many pruning approaches either rely on shape-preserving “fake pruning” or require custom inference engines, limiting real-world applicability. This gap between pruning research and deployment-ready inference systems remains an underexplored challenge.

Mixed-Precision and Sensitivity-Aware Quantization.

Uniform quantization across all layers is suboptimal due to varying sensitivity of transformer components to low-bit precision. Prior work has shown that selectively allocating higher precision to sensitive layers can improve accuracy under tight memory budgets. Techniques based on activation statistics, output error, or second-order information have been explored to guide mixed-precision allocation (Lin et al., 2024; Liu et al., 2024). Many of these models introduce an additional complexity that requires more modifications to the graph. This in-turn makes them less suitable for edge deployment.

However, most approaches either assume uniform model architectures or do not fully address deployment compatibility with existing inference frameworks. In contrast, this work focuses on a system-aware compression pipeline that jointly considers exact pruning, activation-aware quantization, and inference framework constraints. By modifying llama.cpp to support non-uniformly pruned models at runtime, we bridge the gap between LLM compression research and practical deployment on CPUs and embedded devices.

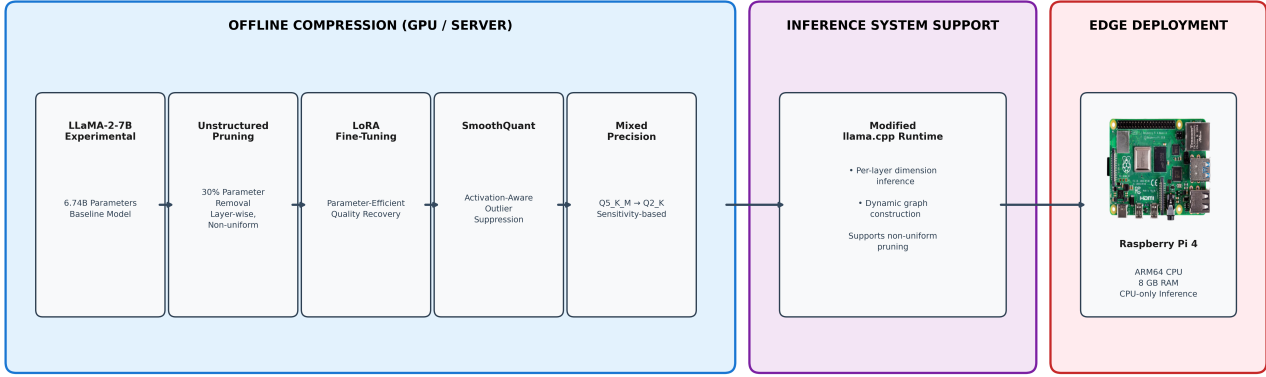


Figure 1. End-to-end compression and deployment pipeline. The model is first pruned using exact unstructured pruning, followed by LoRA fine-tuning, SmoothQuant calibration, mixed-precision quantization, GGUF conversion, and CPU/edge deployment via a modified llama.cpp supporting non-uniform layer dimensions.

3. Methodology

Our methodology follows a deployment-oriented design. As we already know that algorithmic compression techniques are coupled with inference system constraints. Our complete Pipeline consists of unstructured pruning, parameter efficient fine tuning and activation aware quantization allocation, and system-level modifications to support inference of non-uniformly pruned models in llama.cpp.

3.1. Exact Unstructured Pruning

We apply unstructured pruning to the attention and feed-forward layers of LLaMA-2-7B. We used magnitude-based importance scores. Given a weight matrix $\mathbf{W}$, pruning is performed by removing parameters with the smallest L2 magnitude:

$$\mathbf{W}' = \mathbf{W} \odot \mathcal{K}_{|\mathbf{W}| > \tau}, \quad (1)$$

where τ is selected to achieve a target pruning ratio of 30%. Unlike the shape preserving approaches, exact pruning removes parameters, resulting in reduced tensor dimensions. Overall, Pruning is applied in a layer-wise adaptive manner. This allows different transformer blocks to retain different number of attention heads and neurons.

This pruning strategy significantly reduces parameter count and computational cost but produces non-uniform layer dimensions across transformer blocks. These are incompatible with the standard inference engines that assume uniform architectures.

3.2. Parameter-Efficient Fine-Tuning with LoRA

pruning introduces degradation in language modeling performance. Now we address the problem to recover the lost capacity without increasing memory overhead. For this we

apply Low-Rank Adaptation (LoRA). For a linear projection $\mathbf{W}_0$, LoRA introduces a low-rank update:

$$\mathbf{h} = \mathbf{W}_0 \mathbf{x} + \frac{\alpha}{r} \mathbf{B} \mathbf{A} \mathbf{x}, \quad (2)$$

where $\mathbf{A} \in \mathbb{R}^{r \times d}$ and $\mathbf{B} \in \mathbb{R}^{d \times r}$ are trainable matrices, r is the rank, and α is a scaling factor. We use higher LoRA ranks for more aggressively pruned models (up to $r = 64$ for 30% pruning) to improve recovery.

Fine-tuning is performed on the Alpaca-cleaned dataset for 2–3 epochs with a learning rate of 10^{-4} . Next we ensured stability, by employing aggressive gradient clipping, NaN detection, mixed-precision training, and gradient checkpointing.

3.3. Activation-Aware Quantization with SmoothQuant

Post-training quantization is critical for reducing memory footprint and improving CPU inference speed. However, naive low-bit quantization suffers from severe accuracy degradation due to activation outliers. To address this, we adopt SmoothQuant, which redistributes quantization difficulty between activations and weights by applying per-channel scaling:

$$\mathbf{Y} = \mathbf{X} \text{diag}(\mathbf{s})^{-1} \cdot \text{diag}(\mathbf{s}) \mathbf{W}, \quad (3)$$

where the scaling factors are computed as:

$$s_j = \frac{\max(|\mathbf{X}_j|)^\alpha}{\max(|\mathbf{W}_j|)^{1-\alpha}}. \quad (4)$$

We set $\alpha = 0.85$ and calibrate using 512 samples from WikiText-2. SmoothQuant enables stable INT8 and lower-bit quantization without modifying the forward pass, making it suitable for deployment.

3.4. Sensitivity-Based Mixed Precision Quantization

Different transformer layers exhibit varying sensitivity to low-bit quantization. To exploit this, we compute a layer-wise sensitivity metric based on the mean squared error between FP16 and quantized outputs:

$$\mathcal{S}_\ell = \text{MSE}(\mathbf{Y}_\ell^{\text{FP16}}, \mathbf{Y}_\ell^{\text{Q}}). \quad (5)$$

Layers are ranked by sensitivity and assigned different quantization formats (Q5_K_M, Q4_K_M, Q3_K, Q2_K) under a fixed memory budget equivalent to uniform Q4_K_M. This approach improves accuracy while maintaining or reducing overall model size.

3.5. System-Level Support for Non-Uniform Pruned Models

Exact pruning produces transformer blocks with heterogeneous dimensions, which are unsupported by standard llama.cpp. To enable inference, we modify llama.cpp to dynamically infer per-layer dimensions at runtime by reading GGUF tensor metadata. For each layer, attention head counts and feed-forward dimensions are computed directly from tensor shapes rather than relying on global configuration parameters.

These inferred dimensions are propagated to the internal hyperparameter arrays used by the computation graph builder, allowing attention and feed-forward operations to be constructed with layer-specific sizes. This modification preserves backward compatibility with unpruned models and requires no changes to the GGUF format.

4. Results

We evaluate the proposed compression pipeline across multiple dimensions that are critical for practical deployment: language modeling quality, inference throughput, latency characteristics, memory footprint, and storage efficiency. All results are reported for LLaMA-2-7B using CPU-only inference via llama.cpp, unless stated otherwise.

4.1. Effect of Exact Unstructured Pruning

We started by analyzing the impact of exact unstructured pruning on model quality and inference efficiency prior to quantization. Unlike fake or shape-preserving pruning, exact pruning physically removes parameters and produces non-uniform layer dimensions across transformer blocks.

Pruning leads to consistent throughput improvements due to reduced compute and memory pressure. At 30% pruning, CPU throughput increases from 36.8 to 45.9 tokens per second, corresponding to a $1.25\times$ speedup over the FP16 baseline. This gain comes with a moderate increase in perplexity, motivating the use of parameter-efficient fine-

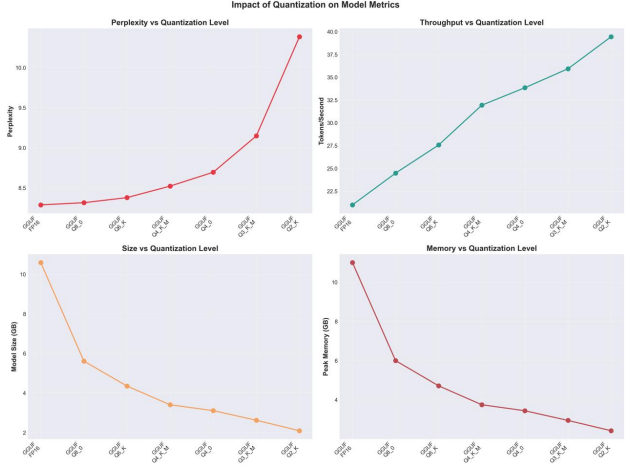


Figure 2. Effect of GGUF quantization formats on perplexity, throughput, model size, and memory footprint for the 30% pruned model.

tuning to recover language modeling performance.

4.2. Impact of Quantization on Accuracy–Efficiency Trade-offs

Although pruning improves inference speed and reduces memory pressure, the resulting models remain impractical for deployment on highly constrained hardware such as Raspberry Pi due to their storage and memory requirements. Quantization is therefore a critical step in enabling real-world deployment. However, aggressive low-bit quantization introduces non-trivial trade-offs between model quality and system efficiency.

Figure 2 summarizes the effect of GGUF quantization formats on perplexity, throughput, memory usage, and model size for the 30% pruned model. As expected, lower-bit quantization monotonically reduces memory footprint and disk size. Moderate formats such as Q4_K_M achieve a favorable balance, retaining reasonable language modeling quality while enabling significant reductions in memory and storage. Extremely aggressive formats such as Q2_K further reduce resource requirements but incur sharper degradation in perplexity.

4.3. Latency Characteristics: Prefill vs. Decode

While throughput provides a coarse measure of inference efficiency, it does not fully capture user-perceived responsiveness, particularly for interactive and edge deployments. In practice, inference latency can be decomposed into two distinct stages: (i) the prefill phase (time-to-first-token), and (ii) the decode phase governing per-token generation cost.

Pruning and quantization primarily reduce decode latency, while prefill latency remains dominated by attention and

Table 1. Summary of quality and system-level metrics across compression stages.

Model	PPL ↓	Speed (tok/s) ↑	Memory (GB) ↓	Size (GB) ↓	Size Red.
Base FP16	7.18	36.8	14.99	25.10	–
Pruned (30%)	7.92	45.9	12.09	20.21	19.5%
GGUF Q8_0	8.31	24.5	6.00	5.61	77.6%
GGUF Q4_K_M	8.52	31.9	3.75	3.41	86.4%
GGUF Q2_K	10.38	39.4	2.43	2.10	91.7%

initial memory access. Aggressive compression reduces both components, leading to faster response times on CPU-only systems and improved interactivity.

4.4. End-to-End Compression and Deployment Results

Finally, we evaluate the complete compression pipeline, integrating exact pruning, activation-aware quantization, mixed-precision allocation, and system-level inference support for non-uniform layer dimensions.

Table 1 summarizes the end-to-end results. The final Q4_K_M model reduces disk size from 25.1 GB to 3.41 GB and peak memory usage from 15.0 GB to 3.75 GB, while maintaining competitive throughput and acceptable perplexity degradation. Notably, the compressed models can be executed entirely on CPUs and fit within the memory constraints of embedded devices such as Raspberry Pi, validating the practicality of the proposed approach.

5. Discussion

Exact Pruning vs. Shape-Preserving (Fake) Pruning.

A key design choice in this work is the use of exact unstructured pruning, in contrast to the commonly adopted shape-preserving or “fake” pruning approaches used in prior work. Fake pruning zeros out weights while maintaining original tensor shapes, which simplifies downstream tooling and avoids changes to inference kernels. However, such approaches do not yield proportional reductions in memory footprint, disk size, or runtime compute, and therefore offer limited benefits for real-world deployment, particularly on CPU-only and embedded systems.

Now comparing this with exact pruning. It physically removes parameters and reduces tensor dimensions, which in turn enables true reductions in compute and memory usage. Our results demonstrate that this distinction is important: while fake pruning may preserve benchmark accuracy, it has a major drawback that it fails to address the fundamental deployment bottlenecks of large language models.

System-Level Challenges Introduced by Exact Pruning. Exact pruning introduces non-trivial system-level challenges. One of these challenges involve removing pa-

rameters because it leads to non uniform layer dimensions across transformer blocks. This violates the uniform-shape assumptions made by most inference engines, including llama.cpp. In conclusion, the pruned models are not compatible with the existing deployment frameworks without adding any effort

We address this challenge by modifying llama.cpp to dynamically infer per-layer tensor dimensions at runtime from GGUF metadata. This enables inference over non-uniformly pruned models without altering model formats or introducing specialized model variants. Importantly, this approach preserves backward compatibility and avoids changes to the computational graph, allowing pruned models to integrate seamlessly into existing inference pipelines.

Role of Quantization in Enabling Edge Deployment.

Pruning alone, even when exact, is insufficient to enable deployment on highly constrained hardware such as Raspberry Pi. Quantization remains a necessary complementary technique to reduce both memory footprint and storage size. However, aggressive low-bit quantization can amplify the sensitivity introduced by pruning, particularly in attention and projection layers.

Our results show that activation-aware techniques such as SmoothQuant are essential for stabilizing low-bit inference in pruned models. Moreover, sensitivity-based mixed precision allocation provides a better accuracy–size trade-off than uniform quantization, especially under strict memory budgets. These findings highlight the importance of jointly considering pruning and quantization rather than treating them as independent steps.

Limitations and Future Directions. Despite its effectiveness, our approach has several limitations. First, runtime support for non-uniform layers introduces modest engineering complexity and may limit compatibility with highly optimized vendor-specific kernels. Second, we focus primarily on perplexity and system-level metrics; downstream task evaluation and long-context performance remain open directions. Finally, while we demonstrate feasibility on Raspberry Pi-class hardware, further optimizations such as operator fusion and kernel-level tuning could yield additional gains.

Future work may explore tighter integration between pruning-aware model design and inference engines, as well as automated sensitivity estimation for mixed precision allocation. Extending the approach to larger models and longer context lengths also remains an important direction.

Implications for Practical LLM Deployment. Conclusively this work demonstrates that deployment of billion-parameter LLMs on CPUs and edge devices requires moving beyond shape-preserving approximations. Furthermore, Exact Pruning when we combine with quantization and runtime support enables real reductions in compute, memory, and storage. Our results suggest that future compression research should place greater emphasis on end-to-end deployability rather than isolated algorithmic gains.

6. Conclusion

This work presents a deployment-oriented compression pipeline that enables billion-parameter large language models to run efficiently on CPU-only and embedded hardware. By combining exact unstructured pruning, activation-aware quantization, mixed-precision allocation, and system-level inference support, we demonstrate that practical deployment of LLaMA-2-7B on constrained devices is feasible without prohibitive degradation in language modeling quality.

A key contribution of this work is moving beyond shape-preserving “fake” pruning toward exact pruning, which delivers real reductions in compute, memory, and storage. We show that while exact pruning introduces non-uniform layer dimensions that are unsupported by standard inference engines, lightweight modifications to llama.cpp enable seamless inference by dynamically inferring per-layer dimensions at runtime. This approach preserves backward compatibility and avoids changes to model formats or computational graphs.

Our experimental results demonstrate substantial system-level gains: up to $1.25\times$ throughput improvement from pruning alone, and up to 91.7% reduction in disk size when combined with low-bit GGUF quantization.

Overall, this work bridges the gap between compression research and real-world deployment, we showed that meaningful efficiency gains require end-to-end consideration of both model structure and runtime constraints. We hope this study encourages future work to prioritize deployability alongside accuracy in the design of large language model compression techniques and this work can pave the way for future deployable breakthroughs.

Acknowledgments

We thank Muhammad Ahmad for supervision and guidance throughout this project. We acknowledge Modal Labs for providing cloud GPU infrastructure and the open-source community for LLMPruner, SmoothQuant, and llama.cpp frameworks.

References

- Gerganov, G. llama.cpp: Llm inference in c/c++. <https://github.com/ggml-org/llama.cpp>, 2023.
- Hugging Face. Introduction to ggml. <https://huggingface.co/blog/ggml>, 2023.
- Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. Awq: Activation-aware weight quantization for llm compression and acceleration. In *Proceedings of the MLSys Conference*, 2024.
- Liu, Z., Zhao, C., Fedorov, I., Soran, B., Choudhary, D., Krishnamoorthi, R., Chandra, V., Tian, Y., and Blankevoort, T. Spinqant: Llm quantization with learned rotations. *arXiv preprint arXiv:2405.16406*, 2024.
- Ma, X., Fang, G., and Wang, X. Llm-pruner: On the structural pruning of large language models. *arXiv preprint arXiv:2305.11627*, 2023.
- Shao, W., Chen, M., Zhang, Z., Xu, P., Zhao, L., Li, Z., Zhang, K., Gao, P., Qiao, Y., and Luo, P. Omniquant: Omnidirectionally calibrated quantization for large language models. *arXiv preprint arXiv:2308.13137*, 2023.
- Xiao, L., Lin, J., and Han, S. Smoothquant: Accurate and efficient post-training quantization for large language models. *arXiv preprint arXiv:2211.10438*, 2022.

A. Additional Experimental Results

This appendix presents detailed experimental results and diagnostic visualizations that complement the quantitative analysis in the main paper. These figures provide fine-grained insights into inference performance, memory usage, latency behavior, and quantization trade-offs, and are included here to preserve clarity in the main text.

A.1. Perplexity Across Compression and Quantization Stages

Figure 3 reports language modeling perplexity on WikiText-2. Exact pruning introduces moderate degradation, while lower-bit quantization causes non-linear increases in perplexity, particularly below 4-bit precision.

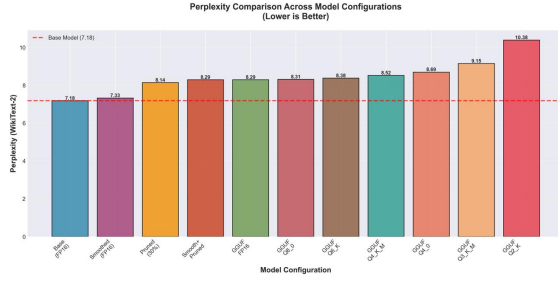


Figure 3. Perplexity comparison across baseline, pruned, and GGUF-quantized models (lower is better).

A.2. Model Storage Size Reduction

Figure 4 illustrates the reduction in on-disk model size achieved through pruning and quantization. GGUF quantization enables up to 91.7% storage reduction relative to the FP16 baseline.

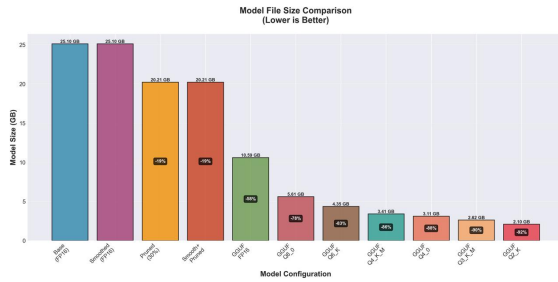


Figure 4. Model file size comparison across compression configurations (lower is better).

A.3. Peak Memory Consumption During Inference

Figure 5 reports peak RAM usage during inference. Quantized GGUF models substantially reduce memory footprint, enabling deployment on memory-constrained edge devices such as Raspberry Pi.

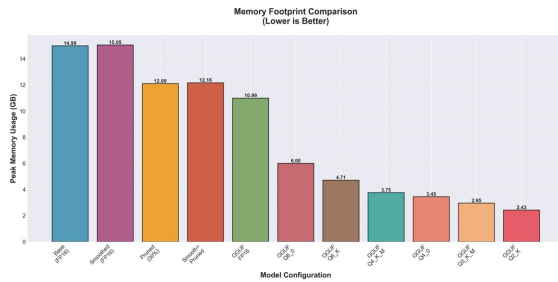


Figure 5. Peak memory usage during inference for different compression settings.

A.4. Inference Speedup Relative to Baseline

Figure 6 normalizes inference throughput relative to the FP16 baseline. While pruning provides the largest speedup, extreme quantization recovers speed at the expense of accuracy.

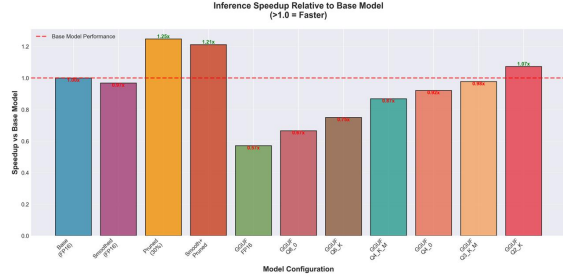


Figure 6. Inference speedup relative to the FP16 baseline (> 1 indicates faster).

A.5. Latency Breakdown: Prefill vs. Decode

Figure 7 decomposes end-to-end latency into prefill (time-to-first-token) and decode stages. Quantization primarily reduces decode latency, while pruning improves both stages by lowering compute intensity.

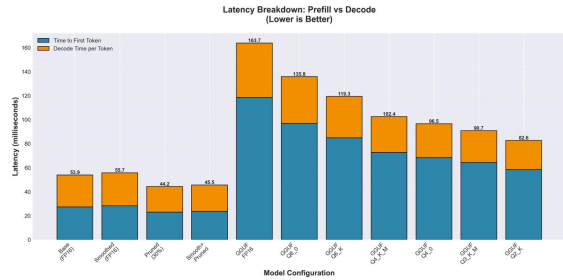


Figure 7. Latency breakdown into prefill and decode stages (lower is better).